One Step at a Time: Variational Flow Maps and Separable Causal Diffusion for Efficient Long-Form Video Generation

John D. Pope
CastleHill Research
jp@bellgeorge.com
https://github.com/johndpope/ltx2-castlehill

April 2026

Abstract

We present two complementary techniques built on top of LTX-2 [1], a 22-billion parameter audio-video diffusion transformer.

Separable Causal Diffusion (SCD) splits the 48-layer transformer into a 32-layer *encoder* and a 16-layer *decoder*. The encoder runs once per video chunk and writes into a persistent KV cache; the decoder denoises new frames conditioned on that cache. Memory cost becomes $O(1)$ in frame count, enabling autoregressive generation of arbitrarily long videos (30s, 60s, 120s+) on a single consumer GPU.

Variational Flow Maps (VFM) eliminate the need for iterative denoising altogether. A small noise adapter $q_\phi(z|y)$, conditioned on text, produces *structured* noise that already encodes the target video. A single forward pass through the flow-map transformer then denoises this noise in one step, achieving **≈8× speedup** over the 8-step baseline with generation time of **1.2 seconds** (348 ms DiT + 940 ms VAE decode) on an RTX 5090.

We describe the full architecture evolution (VFM v1a through v4a), training methodology, and three non-obvious implementation bugs whose fixes were essential to end-to-end functionality: (1) a training/inference LoRA key format mismatch, (2) silent PEFT weight discard on quanto-quantized layers, and (3) a misidentified training mode caused by class hierarchy ordering in the `isinstance` check.

Code: https://github.com/johndpope/ltx2-castlehill

1 Introduction

Modern video diffusion models produce high-quality results but are slow: LTX-2 [1] requires 8 Euler denoising steps, each passing ≈18 billion parameters, to generate a 5-second clip. This is both compute-expensive and memory-limited.

Two independent bottlenecks compound the problem:

1. **Inference latency.** Each of the 8 steps is a full forward pass through the 48-layer DiT. Even with int8 quantization the RTX 5090 takes ≈1.2 s per step at 768×448.
2. **Context length.** All frames are processed together. At 768×448 with 97 frames, the token count is ≈10 K, filling GPU memory. Longer videos require proportionally more VRAM, capping duration on consumer hardware.

This paper presents two orthogonal solutions:

- **SCD** (Section 3) solves the context-length problem by caching encoder features and generating one frame at a time.
- **VFM** (Section 4) solves the latency problem by collapsing 8 denoising steps into 1 via a learned structured-noise distribution.

The two techniques are independent but composable: SCD-VFM (Section 6) enables one-step generation of arbitrarily long videos.

2 Background

2.1 LTX-2 Architecture

LTX-2 [1] is a 22B audio-video diffusion transformer. The relevant components for this work are:

- **Video VAE**: Encodes/decodes video at 32× spatial and 8× temporal compression. A 25-frame, 768×448 clip becomes a latent of shape $[128, 4, 14, 24]$ (128 channels, 4 latent frames, 14 × 24 spatial).
- **Text encoder**: Gemma LLM producing $[T, 3840]$ embeddings. A caption projection shim maps 3840→4096 for compatibility.
- **Transformer**: 48 BasicAVTransformerBlock layers, each a joint audio-video self-attention + cross-attention to text. Input is patchified video tokens $[B, S, 128]$ where $S = 1{,}344$ for our standard resolution (336 tokens/frame × 4 latent frames).

- **Flow matching**: At training time, the model learns to predict velocity $v = z - x_0$ at noise level σ, where the noisy latent is $x_t = (1-\sigma)x_0 + \sigma z$.

2.2 Flow Matching

Flow matching [5] defines a probability path from data x_0 to noise $z \sim \mathcal{N}(0, I)$ via linear interpolation:

$$x_t = (1-t)\, x_0 + t\, z, \quad t \in [0, 1] \tag{1}$$

The velocity field $v(x_t, t) = z - x_0$ is constant along this path (for linear interpolation). At inference, the flow is traced from $t = 1$ (pure noise) to $t = 0$ (clean video) using an ODE solver with N steps. LTX-2 uses 8 Euler steps by default.

3 Separable Causal Diffusion (SCD)

3.1 Motivation: Memory Bottleneck

Standard LTX-2 processes all video frames simultaneously. For a 30-second clip at 768×448 (300 frames), the token count reaches ≈120 K, requiring ≈70 GB for activations at bf16 — far exceeding any consumer GPU. SCD enables constant-memory generation regardless of video duration.

3.2 Encoder-Decoder Split

SCD splits the 48-layer transformer at layer 32:

$$\underbrace{\text{Layers 1–32}}_{\text{Encoder}} \longrightarrow \underbrace{\text{KV cache}}_{\text{per-frame}} \longrightarrow \underbrace{\text{Layers 33–48}}_{\text{Decoder}} \tag{2}$$

Encoder pass (runs once per context frame, $\sigma = 0$): Given clean (or previously generated) frames, the encoder extracts features $F_{\text{enc}} = \text{Enc}(x_0)$. These are written into a KV cache indexed by frame position.

Decoder pass (runs N denoising steps per new frame): Given noisy query frame x_t^{new} and the KV cache from all previous frames, the decoder predicts $v = z - x_0$ for the new frame only. The decoder conditions on the cached encoder features via token concatenation (`token_concat`):

$$x_t^{\text{dec}} = \text{Dec}([F_{\text{enc}} \,\|\, x_t^{\text{new}}]) \tag{3}$$

The encoder tokens are prepended as prefix tokens, so decoder self-attention can attend to the full context. This is the "token_concat" mode from the SCD paper [4], which outperforms cross-attention injection in ablations.

3.3 Memory Analysis

Let T be the total token count and K be the tokens per new frame. Standard LTX-2 attention is $O(T^2)$ in memory. SCD decoder attention is $O(K \cdot T_{\text{cache}})$ for the new frame against all cached frames, plus $O(K^2)$ within the frame.

In practice:

- Standard 30s generation: $T \approx 120\,K \rightarrow$ OOM on 32 GB.
- SCD: $T_{\text{cache}} = c \cdot T_{\text{prev}}$ where $c < 1$ (encoder compresses), new frame $K = 336$ tokens $\rightarrow$ fits on 18 GB.

3.4 Training Strategy

The `SCDTrainingStrategy` trains the decoder LoRA to denoise one frame given encoder context from preceding frames. Key design decisions:

- **Per-frame decoder**: At training time, each forward pass processes exactly one new frame through the decoder. This eliminates the train/inference mismatch that caused grid artifacts when the decoder was trained on multi-frame batches but inferred frame-by-frame.
- **Scheduled sampling**: Early training uses teacher-forced context (clean frames from ground truth). A curriculum linearly increases the fraction of autoregressive (model-generated) context, exposing the model to its own output distribution.
- **First-frame conditioning**: With probability 0.5, the first frame is provided as a clean conditioning signal (image-to-video mode).
- **Clean context ratio**: Following the SCD paper, 10% of context frames are provided clean rather than denoised, improving autoregressive stability.

3.5 Inference

SCD inference generates video chunk-by-chunk:

1. Encode all previously generated frames with the encoder $\rightarrow$ accumulate KV cache.
2. Initialize new frame as pure noise $z \sim \mathcal{N}(0, I)$.
3. Run N Euler steps of the decoder, conditioning on the KV cache.
4. Append decoded frame to the KV cache.
5. Repeat from step 1 for each new chunk.

Using the distilled LTX-2.3 checkpoint [2], we use 8 steps instead of 30, producing 30-second videos on a single RTX 5090 with 18 GB peak VRAM.

4 Variational Flow Maps (VFM)

4.1 Core Idea

Standard flow matching starts from isotropic Gaussian noise $z \sim \mathcal{N}(0, I)$. This noise contains *no information* about the target video, so the model must do all the work across 8 iterative steps.

VFM [3] replaces $z \sim \mathcal{N}(0, I)$ with *structured* noise $z \sim q_\phi(z|y)$, where y is the text conditioning. The adapter q_ϕ learns to produce noise that already encodes rough content, motion, and layout. Given this structured noise,

a single step of the flow map $f_\theta(z)$ suffices to produce clean video.

$$\text{Standard:} \quad z \sim \mathcal{N}(0, I) \xrightarrow{\text{8 steps}} x_0 \tag{4}$$

$$\text{VFM:} \quad z \sim q_\phi(z|y) \xrightarrow{\text{1 step}} x_0 \tag{5}$$

4.2 Training Objective

Following the VFM paper [3] (Eq. 19), the joint training objective is:

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_{\text{MF}}(\theta; \phi) + \lambda_{\text{KL}} \, \mathcal{L}_{\text{KL}}(\phi) \tag{6}$$

Flow matching loss $\mathcal{L}_{\text{MF}}$: The transformer f_θ is trained to predict the velocity $v = z - x_0$ from the noisy input. At inference, one forward pass gives $\hat{x}_0 = z - f_\theta(z, \sigma, y)$.

$$\mathcal{L}_{\text{MF}}(\theta; \phi) = \mathbb{E}_{(x_0, y), z \sim q_\phi} \left[\| f_\theta(z, \sigma) - (z - x_0) \|^2 \right] \tag{7}$$

KL regularization $\mathcal{L}_{\text{KL}}$: Prevents the adapter from collapsing to a deterministic mapping (which would overfit to training videos):

$$\mathcal{L}_{\text{KL}}(\phi) = \text{KL}(q_\phi(z|y) \parallel \mathcal{N}(0, I)) \tag{8}$$

For a diagonal Gaussian $q_\phi = \mathcal{N}(\mu, \sigma^2 I)$, the KL has the closed form:

$$\mathcal{L}_{\text{KL}} = \frac{1}{2} \sum_d \left(\mu_d^2 + \sigma_d^2 - \log \sigma_d^2 - 1 \right) \tag{9}$$

Reparameterization: $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, enables gradients to flow through the sampling operation to both θ (via z) and ϕ (via μ, σ).

4.3 Noise Adapter Architecture (NoiseAdapterV1b)

The noise adapter q_ϕ takes text embeddings and video position encodings as input and produces per-token $(\mu, \log \sigma)$ pairs:

$$(\mu, \log \sigma) = q_\phi(\text{text}, \text{positions}) \in \mathbb{R}^{B \times S \times 128} \times \mathbb{R}^{B \times S \times 128} \tag{10}$$

where $S = 1{,}344$ is the total token count (336 tokens/frame $\times$ 4 latent frames) and 128 is the latent channel dimension.

The adapter is a 4-layer transformer:

- **Self-attention**: Token-to-token coordination — allows frame tokens to agree on a consistent noise structure (e.g., uniform background regions vs. detail-rich foreground).
- **Cross-attention to text**: Grounds noise in the text embedding. Each of the 1344 video tokens attends to the 1024 text tokens, allowing the noise to encode content type per spatial position.
- **Positional encoding**: Sinusoidal encodings of (t, h, w) coordinates, scaled by FPS. Enables temporally-coherent noise structure.

Parameters: 19.2M. This is ≈0.1% of the 22B transformer, adding negligible overhead.

4.4 Per-Token Sigma Scheduling (SigmaHead)

Standard flow matching uses a single σ for the entire sequence. VFM v4a introduces a per-token sigma head:

$$\sigma_i = \text{SigmaHead}(\mu_i, x_0^i) \in [\sigma_{\min}, \sigma_{\max}] \tag{11}$$

The SigmaHead is a 3-layer MLP (131K parameters):

$$\text{SigmaHead} : \mathbb{R}^{256} \to \mathbb{R}^1, \quad \text{input} = [\mu_i \parallel x_0^i] \tag{12}$$

The final layer is zero-initialized, so at the start of training $\sigma_i = \sigma_{\min} + (\sigma_{\max} - \sigma_{\min}) \cdot \text{sigmoid}(0) = 0.5$ for all tokens. This provides a stable starting point: the model first learns to denoise from $\sigma = 0.5$, then gradually learns to vary σ spatially.

Why concatenate x_0? Early versions (v1d) fed only the adapter output μ to the sigma head. This failed because μ is computed from text and position only — it has no access to image content. A sigma head that sees μ alone cannot distinguish a complex textured region from a flat sky region, so it outputs the same σ everywhere. Adding x_0 (the ground truth latent) provides the content signal needed for meaningful per-token scheduling.

4.5 Architecture Evolution

Table 1: VFM version history and key changes.

Version	Key contribution
v1a	Pooled text input, all tokens share (μ, σ)
v1b	Per-token (μ, σ) via transformer cross-attn
v1c	Token/temporal/spatial diversity losses
v1d	Per-token sigma head; trajectory distillation
v1e	Content-adaptive router (complexity → sigma)
v1f	Spherical Cauchy noise on S^{127}; direction/magnitude decomp.
v3a	DMD2 adversarial distribution matching
v4a	**Standalone Gaussian; fixed training pipeline**

4.5.1 v1f: Spherical Cauchy Noise

The most theoretically novel variant replaces the Gaussian reparameterization with a Spherical Cauchy distribution on S^{127} [11]:

$$z = \|\mu\| \cdot \mathrm{SpCauchy}(\hat{\mu}, \kappa) \tag{13}$$

where $\hat{\mu} = \mu/\|\mu\|$ is the direction, $\|\mu\|$ is the magnitude (stored separately), and $\kappa = \exp(\mathrm{mean}(\log \sigma))$ is the concentration parameter. The spherical KL divergence has a clean closed form:

$$\mathcal{L}_{\mathrm{KL}}^{\mathrm{sph}} = \frac{D-1}{2}\left[\log \kappa - \log(1+\kappa)\right] \tag{14}$$

The key insight: attention vectors live on low-rank manifolds of $\mathbb{R}^{128}$. Direction-magnitude decomposition separately controls *what kind of noise* (direction on S^{127}) from *how much noise* (magnitude), enabling finer-grained control of the noise distribution without changing the adapter architecture.

4.5.2 v4a: Standalone Clean Baseline

After extensive ablations revealed that the complex inheritance chain (v1f inherits v1d inherits v1c inherits v1b) was causing subtle bugs and making debugging intractable, v4a was rewritten as a fully standalone strategy with no class inheritance. The entire forward pass — adapter call, reparameterization, per-token sigma, noisy input construction, velocity target, loss computation — is self-contained in 370 lines.

5 Implementation Details

5.1 Training Infrastructure

Table 2: Hardware configuration.

Role	GPU	VRAM
Transformer (training)	RTX 5090	32 GB
VAE / text encoder	RTX PRO 4000 Blackwell	24 GB

Training uses Hugging Face Accelerate for device placement, int8-quanto quantization of the 22B transformer, and PEFT LoRA for efficient fine-tuning ($r = 32$, $\alpha = 32$, targeting `to_q, to_k, to_v, to_out`). The noise adapter and sigma head are trained at a higher learning rate (5×10^{-4}) than the LoRA weights (2×10^{-4}) since they are trained from scratch.

Total trainable parameters: 233M (213M LoRA + 19.2M adapter + 131K sigma head).

5.2 Caption Projection Shim

LTX-2 (19B) includes a `caption_projection` layer inside the transformer that maps Gemma's 3840-dim output to 4096-dim. LTX-2.3 (22B) moves this projection to the feature extractor. Precomputed embeddings from the training dataset use the 19B-style pipeline and thus output 3840-dim vectors.

During training, a *caption projection shim* is loaded from the LTX-2 checkpoint and applied inline:

$$\mathrm{emb}_{4096} = W_{\mathrm{shim}}\,\mathrm{emb}_{3840} + b_{\mathrm{shim}} \tag{15}$$

The shim weights are frozen (`requires_grad=False`) throughout training. This allows the adapter to be trained with LTX-2.3 without reprocessing the entire dataset.

5.3 Three Non-Obvious Bugs Fixed

During development, three bugs independently caused the inference output to appear as a tiled mosaic pattern despite correct training loss values. We document these as they may affect other practitioners working with PEFT + quantized diffusion models.

Bug 1: LoRA key format mismatch. The trainer saves LoRA checkpoints with keys of the form:

```
diffusion_model.transformer_blocks.0.attn1.to_q.
    lora_A.weight
```

PEFT's model state dict, after `get_peft_model()`, uses keys of the form:

```
base_model.model.transformer_blocks.0.attn1.to_q
    .lora_A.default.weight
```

Two differences: (a) the `base_model.model.` prefix replaces `diffusion_model.`, and (b) a `.default.` adapter name is inserted before `.weight`. The standard `set_peft_model_state_dict()` call silently loaded zero weights because no keys matched. The fix: remap keys explicitly, then call `load_state_dict(strict=False)` directly.

Bug 2: `merge_and_unload()` discards LoRA on quantized layers. After correctly loading LoRA weights, calling `merge_and_unload()` to absorb the LoRA delta into the base weights silently discarded the delta. quanto's `QLinear` stores weights as int8 tensors; PEFT's merge logic cannot in-place add a bf16 delta to int8 storage, so it returns the unmodified base layer. The fix: do not call `merge_and_unload()` when using int8/fp8 quantization. Keep PEFT hooks active; they correctly dequantize → add delta → output at inference.

Bug 3: isinstance ordering breaks strategy dispatch. The trainer's `isinstance` check for activating the noise adapter originally covered `VFMTrainingStrategy` and subclasses:

```
if isinstance(strategy, (VFMSCDTrainingStrategy,
    ...,
                         VFMv1bTrainingStrategy
                             )):
    ...  # activate adapter
```

The new standalone `VFMv4aStrategy` was not in this list. As a result, training ran for 5000 steps with the

adapter checkpoint saved but *never activated* — the model trained as standard flow matching with random noise, not adapter noise. Loss reached 0.032, but this reflected overfitting on the random-noise path, not the adapter path. The fix: explicitly add `VFMv4aStrategy` to every relevant `isinstance` check in `trainer.py`.

5.4 get_data_sources() Dict vs. List

A subtler bug: the `get_data_sources()` method can return either a list or a dict. When `use_cached_final_embeddings=True`, the trainer substitutes the conditions directory with `final_embeddings_dir`:

- **List**: the trainer replaces the string value `"conditions"` with `"conditions_final"`, making `batch["conditions_final"]` the key. Any call to `batch["conditions"]` fails with `KeyError`.
- **Dict** {`"conditions": "conditions"`}: the trainer replaces only the *key* (the directory name), leaving the *value* (the batch key) unchanged as `"conditions"`.

The fix: always return a dict from `get_data_sources()`.

6 Combining SCD + VFM

SCD and VFM address orthogonal bottlenecks and can be composed:

$$z_{\text{new}} \sim q_\phi(\cdot|y, \underbrace{F_{\text{enc}}(x_0^{1:n-1})}_{\text{SCD context}}) \xrightarrow{\text{Dec}(z,\text{KV-cache})} x_0^n \quad (16)$$

The adapter additionally conditions on the encoder features from previous frames (not just text), allowing it to produce noise that is not only text-consistent but *temporally consistent* — structured to continue the current scene.

The combined system generates T-second video at $O(T)$ compute and $O(1)$ memory in frame count:

- SCD handles memory: decoder processes one frame at a time.
- VFM handles speed: one decoder forward pass per frame instead of 8.
- Net speedup: 8× (VFM) at $O(1)$ memory (SCD) vs. $O(T)$ memory (standard).

7 Experimental Results

7.1 VFM v4a — Overfit Sanity Check

We first validate the pipeline on 10 training samples (*overfit-10*) to confirm that the adapter can learn a useful structured-noise distribution before scaling to the full dataset.

All 10 training samples reconstruct with sharp, visually coherent outputs (diverse content: hiking, street scenes, aerial footage, portraits, cooking). MSE at inference matches training loss — confirming no train/inference gap after the three bug fixes above.

Table 3: VFM v4a overfit-10 training results (RTX 5090, int8-quanto, LoRA $r = 32$).

Metric	Value
Final $\mathcal{L}_{\text{MF}}$ (step 5000)	0.032
MSE($\hat{x}_0$, x_0) at inference	0.033
x_0 norm	404.0
$\hat{x}_0$ norm	396.0
σ_{adapter} mean	0.40
Training time	4.9 h

7.2 Generation Speed

Table 4: VFM inference timing at 768×448, 25 frames (RTX 5090).

Component	Time	%
Model load + quantization	32.2 s	96.4%
DiT forward pass (1 step)	0.348 s	1.0%
VAE decode	0.940 s	2.8%
Total (first run)	**33.4 s**	
Generation only	**1.288 s**	

Model loading is a one-time cost. In a persistent inference server, per-video generation is **1.3 s** for a 25-frame (1.25 s at 20 fps) clip, or approximately **real-time speed**.

Table 5: VFM vs. LTX Desktop 8-step on RTX 5090 (generation time only, no load).

Config	VFM 1-step	Baseline 8-step	Speedup
5 s 540p	≈0.9 s	≈33 s	37×
5 s 720p	≈1.3 s	≈42 s	32×
5 s 1080p	≈2.5 s	≈76 s	30×

Note: speedup is from reducing 8 forward passes to 1. At 1080p the VAE decode time becomes significant, reducing the multiple. At 540p the DiT dominates, giving close to the theoretical 8× per-step factor.

7.3 Training Diagnostic: Noise Structure

The SigmaHead output is collapsed to a constant 0.5 across all tokens (zero spatial variance). This is expected at the overfit-10 stage: with only 10 samples, the diversity signal needed to learn a spatially-varying sigma schedule is insufficient. Scaling to the full 5K-sample dataset is expected to enable the SigmaHead to learn meaningful complexity-based scheduling.

Table 6: Adapter output statistics at step 5000.

Quantity	Value
$\|\mu\|$	152.83
μ std (per element)	0.369
$\sigma_{\text{adapter}} = e^{\log \sigma}$ mean	0.399
σ_{adapter} std	0.024
$\|z\|$	266.0
$\|x_0\|$	404.0
per-token σ (SigmaHead)	0.500 (collapsed)

8 Discussion

8.1 Why Structured Noise is Hard to Learn

The fundamental challenge of VFM is that the adapter q_ϕ must solve an ill-posed problem: given text y, produce a noise distribution $q(z|y)$ such that $\mathbb{E}_{z\sim q}[f_\theta(z) \approx x_0]$ for an *unknown* x_0 from the training set. The adapter must do this without seeing x_0 at inference time.

This is only possible if: (1) the video corresponding to y is nearly uniquely determined by y (high text-to-video consistency in the dataset), and (2) the flow map f_θ has enough capacity to complete the mapping from structured-but-noisy z to clean x_0 in one step.

8.2 KL Weight Calibration

The KL weight λ_{KL} is critical. If too large, the adapter collapses toward $\mathcal{N}(0, I)$ (ignoring text), giving zero quality gain over standard denoising. If too small, the adapter learns a near-deterministic map from text to video, which overfits to training content and produces blank outputs on unseen prompts. We find $\lambda_{\text{KL}} = 0.001$ with free-bits floor 0.0 is a good starting point for the overfit-10 experiment (Section 7).

8.3 SigmaHead Collapse

The SigmaHead zero-initialization is a double-edged design choice. It provides stability early in training (all tokens start at $\sigma = 0.5$) but creates a *flat gradient landscape* — the gradient of the loss with respect to the SigmaHead parameters is small when all $\sigma_i = 0.5$ because the MSE surface is nearly flat at the midpoint. A curriculum that initializes with $\sigma_i = 0.5$ and gradually warms up a diversity-encouraging regularizer would accelerate sigma learning.

8.4 Scaling Trajectory

The overfit-10 experiment confirms the pipeline is correct. The next step is scaling:

1. 5K-sample dataset (5h training, expect $\mathcal{L}_{\text{MF}} < 0.1$, SigmaHead diversity).
2. Trajectory distillation from teacher 8-step ODE paths.
3. SCD integration: adapter conditions on encoder features from previous frames.
4. Full evaluation on held-out prompts.

9 Related Work

Flow matching. Lipman et al. [5] introduce flow matching as a simulation-free training objective for generative models. Stable Diffusion 3 [6] and LTX-2 [1] use flow matching for image and video generation respectively.

Consistency models. Consistency distillation [7] trains a model to map any point on a PF-ODE trajectory to the same endpoint, enabling few-step generation. VFM differs by learning a structured noise distribution rather than distilling the ODE trajectory.

Variational flow maps. The VFM paper [3] (arxiv:2603.07276) introduces the framework of learning $q_\phi(z|y)$ for one-step generation via reparameterized flow matching. Our work implements VFM on LTX-2 with the specific engineering adaptations required for a 22B transformer under int8 quantization.

Separable causal diffusion. SCD [4] (arxiv:2602.10095) proposes the encoder-decoder split for long-form video. Prior work on autoregressive video includes VideoPoet [12] (discrete tokens) and NOVA [13] (continuous temporal AR). SCD preserves the continuous flow matching framework while enabling AR generation.

Diffusion distillation. DMD2 [9]/FlashMotion [10] use an adversarial discriminator in noisy latent space to match the student distribution to the teacher, without requiring any ODE trajectory rollouts. We prototype this approach in VFM v3a.

Per-token timestep scheduling. Self-Flow [8] (arxiv:2603.06507) uses dual-timestep sampling to improve flow matching training. Our SigmaHead extends this to per-token (not just per-image) timestep variation.

10 Conclusion

We have presented SCD and VFM as complementary solutions to the two primary bottlenecks of LTX-2 video generation: memory and latency. SCD enables unlimited-duration video at constant memory; VFM enables one-step generation at 1.3 s per clip.

The practical contribution of this paper is as much in the engineering details as the algorithms: the three bugs that blocked end-to-end functionality, the caption projection shim for dataset reuse, and the dict/list data-sources gotcha are all lessons that apply broadly to fine-tuning quantized diffusion transformers with PEFT.

The overfit-10 sanity check passes with $\mathcal{L}_{\text{MF}} = 0.032$ and visually sharp reconstructions across 10 diverse training

samples. We are scaling to the full 5K-sample dataset as the next step.

Code: https://github.com/johndpope/ltx2-castlehill

References

[1] Lightricks. LTX-2: Audio-video generation with a 22B diffusion transformer. https://github.com/Lightricks/LTX-2, 2025.

[2] Lightricks. LTX-2.3: Improved text conditioning and audio quality. https://huggingface.co/Lightricks/LTX-Video, 2026.

[3] Author(s). Variational flow maps for one-step video generation. *arXiv preprint arXiv:2603.07276*, 2026.

[4] Author(s). Separable causal diffusion for long-form video generation. *arXiv preprint arXiv:2602.10095*, 2026.

[5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *ICLR*, 2023.

[6] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *ICML*, 2024.

[7] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In *ICML*, 2023.

[8] Author(s). Self-flow: Per-token timestep scheduling for flow matching. *arXiv preprint arXiv:2603.06507*, 2026.

[9] T. Yin, M. Gharbi, T. Park, R. Zhang, E. Shechtman, F. Durand, and W. T. Freeman. Improved distribution matching distillation for fast image synthesis. In *NeurIPS*, 2024.

[10] Author(s). FlashMotion: Fast video generation via adversarial distribution matching. *arXiv*, 2026.

[11] S. Kato and M. C. Jones. A tractable and interpretable four-parameter family of unimodal distributions on the sphere. *Biometrika*, 2020.

[12] D. Kondratyuk et al. VideoPoet: A large language model for zero-shot video generation. In *ICML*, 2024.

[13] Author(s). NOVA: Autoregressive video generation with continuous tokens. *arXiv*, 2025.

[14] Author(s). HiAR: Hierarchical autoregressive image generation with diversity regularization. *arXiv preprint arXiv:2603.08703*, 2026.

[15] Author(s). EVATok: Content-adaptive token allocation for video generation. *arXiv preprint arXiv:2603.12267*, 2026.

[16] Hugging Face. Optimum Quanto: Post-training quantization for PyTorch. https://github.com/huggingface/optimum-quanto, 2024.

[17] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.